

Perfecto 👍 Vamos a hacer un **resumen completo y estructurado** de los **RA1 y RA2** (Resultados de Aprendizaje) de **Programación de Servicios y Procesos** para **2º DAM**, enfocado **al lenguaje Java con IntelliJ IDEA**.

Este resumen te servirá para **repasar conceptos teóricos y prácticos** que suelen entrar en examen.

RA1: Programación de aplicaciones con varios procesos

“Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.”

◆ **Conceptos clave:**

- **Proceso:** Programa en ejecución con su propio espacio de memoria.
 - **Programación concurrente:** Varios procesos o hilos se ejecutan *simultáneamente o alternadamente* para mejorar la eficiencia.
 - **Programación paralela:** Ejecución simultánea real (en varios núcleos o CPUs).
 - **Programación distribuida:** Ejecución en varios equipos conectados por red.
-

◆ **Resumen por criterios de evaluación:**

a) Características de la programación concurrente y ámbitos de aplicación.

- Permite ejecutar tareas de forma simultánea.
 - Útil en sistemas con tareas independientes o que esperan I/O.
 - Ejemplo: un servidor web que atiende varias peticiones a la vez.
-

b) Diferencias entre programación paralela y distribuida.

Tipo	Descripción	Ventajas	Inconvenientes
------	-------------	----------	----------------

Paralela	Varias tareas en varios núcleos del mismo equipo	Rápido, aprovecha CPU	Requiere sincronización cuidadosa
Distribuida	Tareas en varios equipos conectados	Escalable, tolerancia a fallos	Comunicación compleja, latencias

c) Características de los procesos y su ejecución por el sistema operativo.

- Cada proceso tiene su **PID**, su **espacio de memoria propio** y **estado** (nuevo, listo, en ejecución, bloqueado, terminado).
 - El **SO** gestiona la planificación y el cambio de contexto.
-

d) Caracterización de hilos y relación con procesos.

- Los **hilos (threads)** son *subprocesos* dentro de un mismo proceso.
 - Comparten recursos (memoria, archivos, variables globales).
 - Más ligeros y rápidos que los procesos.
-

e) Clases para crear subprocessos (Java).

- `ProcessBuilder` y `Runtime.getRuntime().exec()` permiten ejecutar procesos del sistema desde Java.

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");  
Process proceso = pb.start();
```

f) Mecanismos para compartir información con subprocessos.

- A través de **Streams** (`InputStream`, `OutputStream`) del proceso.

```
BufferedReader br = new BufferedReader(new  
InputStreamReader(proceso.getInputStream()));
```

g) Mecanismos para sincronizar y obtener valores devueltos.

- Esperar a que un proceso termine:

```
int exitCode = proceso.waitFor();
```

- Capturar su salida o error con `getInputStream()` y `getErrorStream()`.
-

h) Aplicaciones que gestionan varios procesos en paralelo.

Ejemplo: un programa Java que lanza varios procesos externos a la vez (compresiones, cálculos, etc.) y espera sus resultados.

i) Depuración y documentación.

- Usar el **depurador de IntelliJ**.
 - Comentar el código (`/** ... */`), documentar entradas/salidas de procesos y manejo de excepciones.
-

RA2: Programación con varios hilos de ejecución

“Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje.”

♦ Conceptos clave:

- **Hilo:** unidad mínima de ejecución dentro de un proceso.
 - **Multihilo (multithreading):** Permite que un mismo programa ejecute tareas concurrentemente.
 - **Librerías Java:** `Thread`, `Runnable`, `Callable`, `ExecutorService`, `Future`, `synchronized`, `ReentrantLock`, etc.
-

♦ Resumen por criterios de evaluación:

a) Cuándo usar varios hilos.

- Cuando hay tareas que pueden ejecutarse en paralelo.
 - Ejemplo: cargar datos mientras se muestra una barra de progreso.
-

b) Creación, inicio y finalización de hilos.

- Heredando de `Thread`:

```
class MiHilo extends Thread {  
    public void run() { System.out.println("Ejecutando hilo..."); }  
}  
new MiHilo().start();
```

- Implementando `Runnable`:

```
new Thread(() -> System.out.println("Hilo con Runnable")).start();
```

c) Programas con varios hilos.

- Ejemplo: 3 hilos que imprimen mensajes simultáneamente.
- Uso de `ExecutorService` para gestionar varios hilos:

```
ExecutorService pool = Executors.newFixedThreadPool(3);  
pool.submit(() -> tarea());  
pool.shutdown();
```

d) Estados de un hilo.

- **NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.**
- Control con `sleep()`, `wait()`, `notify()`, `join()`.

e) Compartición de información entre hilos.

- Variables compartidas, buffers, colas concurrentes.
- Usar sincronización para evitar condiciones de carrera.

f) Sincronización entre hilos.

- Palabra clave `synchronized`.
- Clases del paquete `java.util.concurrent` como `Semaphore`, `Lock`, `CountDownLatch`.

g) Prioridad de los hilos.

- Establecer prioridad con `setPriority()`, aunque el sistema operativo decide en última instancia.

```
hilo.setPriority(Thread.MAX_PRIORITY);
```

h) Depuración y documentación.

- Usar **breakpoints** en IntelliJ.
- Documentar secciones críticas del código y el control de sincronización.

i) Contexto de ejecución de los hilos.

- Cada hilo tiene su propia **pila de ejecución**, pero comparten **heap** y recursos del proceso.

j) Librerías específicas para multihilo.

- `java.lang.Thread`, `java.util.concurrent.*`, `ForkJoinPool`, `ExecutorService`, `Future`, `Callable`, `BlockingQueue`, etc.
-

k) Problemas derivados de la compartición de información.

- **Condiciones de carrera:** dos hilos modifican una variable a la vez.
 - **Interbloqueo (deadlock):** dos hilos se bloquean esperando recursos.
 - **Inanición (starvation):** un hilo no obtiene tiempo de CPU.
 - Soluciones: sincronización, locks, semáforos y control de prioridades.
-



CONSEJOS PARA EL EXAMEN

- Domina los métodos clave de **Thread**: `start()`, `run()`, `sleep()`, `join()`, `interrupt()`.
- Entiende las diferencias entre proceso e hilo.
- Practica con ejemplos pequeños (productor-consumidor, contador sincronizado, etc.).
- Estudia la API **java.util.concurrent**, sobre todo `ExecutorService`, `Callable`, `Future`.
- Sabe cómo ejecutar procesos externos con `ProcessBuilder`.
- Explica claramente conceptos teóricos (conurrencia, paralelismo, sincronización, estados, deadlock).